

ITUNIVERSITY — MODULE M1 · SÉANCE 4

Jetpack Compose : l'UI comme fonction de l'état

Composables, état, recomposition
Mini-TP « Faire vivre un écran »



XML vs Jetpack Compose : la même carte produit

XML + ACTIVITY — 2 langages, 2 fichiers

```
<!-- res/layout/produit_card.xml -->
<LinearLayout android:orientation="vertical">
    <TextView android:id="@+id/tvNom" .../>
    <TextView android:id="@+id/tvPrix" .../>
</LinearLayout>

// ProduitActivity.kt
val tvNom =
    findViewById<TextView>(R.id.tvNom)
val tvPrix =
    findViewById<TextView>(R.id.tvPrix)
tvNom.text = produit.nom
tvPrix.text = formatPrix(produit.prixKg)

// donnée modifiée ? → resynchroniser
// chaque vue À LA MAIN
```

JETPACK COMPOSE — 1 langage, 1 fonction

```
@Composable
fun ProduitCard(produit: Produit) {
    Column(Modifier.padding(16.dp)) {
        Text(
            produit.nom,
            style = MaterialTheme
                .typography.titleMedium
        )
        Text(produit.prixKg
            ?.let { "$it Ar/kg" }
            ?: "prix non fixé")
    }
}
// état modifié ? → recomposition AUTOMATIQUE
```

On ne décrit plus comment mettre l'écran à jour — on déclare ce que l'écran doit être : l'UI est une fonction de l'état.



Le paradigme : décrire l'écran, pas les mises à jour

1 Un écran = une fonction

Annotée `@Composable` : elle reçoit des données et décrit l'interface pour ces données-là.

2 L'état change → recomposition

Compose rappelle la fonction. Pas de `findViewById`, pas de `setText` — pas d'oubli possible.

3 Un bug disparaît par construction

L'« écran pas à jour », classe de bugs la plus fréquente de l'Android historique, ne peut plus exister.

« L'interface est une photographie de l'état. L'état change ? On reprend la photo. »



Les briques : composables, conteneurs, Modifier

La carte produit — du Kotlin, rien d'autre

```
@Composable
fun ProduitCard(produit: Produit) {
    Card(Modifier.padding(16.dp)) {
        Column(Modifier.padding(16.dp)) {
            Text(
                produit.nom,
                style = MaterialTheme
                    .typography.titleLarge
            )
            Text(produit.prixKg
                ?.let { "$it Ar/kg" }
                ?: "prix non fixé")
        }
    }
}
```

Le Modifier — la chaîne de personnalisation

```
Modifier
    .padding(16.dp)        // espace
    .fillMaxWidth()       // largeur
    .clickable { ... }    // réagir au clic

// L'ORDRE COMPTE :
// padding puis clickable
// ≠ clickable puis padding
// (la zone cliquable change)
```

La null safety de la séance 1 traverse jusque dans l'interface : un prix null ne peut pas s'afficher par accident.



Le Modifier en action : une chaîne, un rendu

ProduitCard — la chaîne complète

```
@Composable
fun ProduitCard(p: Produit, onClick: () -> Unit) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp,
                vertical = 8.dp)
            .clickable { onClick() },
    ) {
        Column(Modifier.padding(16.dp)) {
            Row(Modifier.fillMaxWidth()) {
                Text(p.nom,
                    modifier = Modifier.weight(1f),
                    style = titleLarge)
                Text("${p.stockKg} kg")
            }
            Text(p.prixKg?.let { "$it Ar/kg" }
                ?: "prix non fixé")
        }
    }
}
```

1

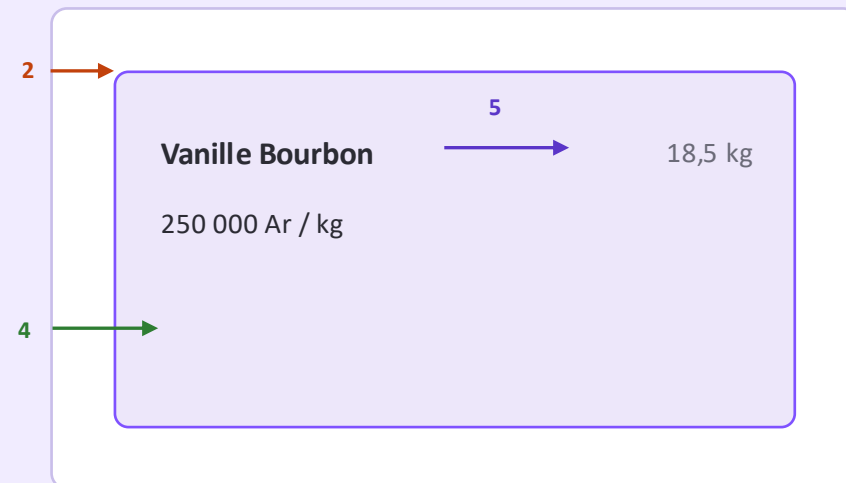
2

3

4

5

Ce que ça donne à l'écran



1

fillMaxWidth — la carte occupe toute la largeur

2

padding (extérieur) — l'espace entre la carte et le bord de l'écran

3

clickable — la carte réagit au toucher

4

padding (intérieur) — l'espace entre le bord de la carte et le texte

5

weight(1f) — le titre prend la place restante → le stock est poussé à droite

L'ORDRE COMPTE : padding AVANT clickable → la marge n'est pas cliquable · padding APRÈS clickable → la marge le devient. Chaque maillon enveloppe le précédent.



L'état : remember + mutableStateOf

Le compteur du mini-TP

```
@Composable
fun ProduitCard(produit: Produit) {
    Log.i("RECOMP", "ProduitCard se (re)compose")

    var quantite by remember { mutableStateOf(0) }

    Column {
        Text("Quantité : $quantite kg")
        Button(onClick = { quantite++ }) {
            Text("Ajouter 1 kg")
        }
    }
}
```

Les trois morceaux

- `mutableStateOf(0)` : un état OBSERVABLE — Compose sait qui en dépend
- `remember` : survit aux recompositions (sans lui : retour à 0 à chaque fois)
- `by` : délégation Kotlin — se lit et s'écrit comme une variable

```
// Fonctionne, mais plus verbeux
val quantite = remember { mutableStateOf(0) }

Text("Quantité : ${quantite.value} kg")
Button(onClick = { quantite.value++ }) {
    Text("Ajouter 1 kg")
}
```

remember survit aux recompositions — pas à la rotation (séance 3). La vraie survie a un nom : ViewModel, séance 6.



Les listes : LazyColumn

Mille produits, le prix de dix

```
@Composable
fun ListeProduits(produits: List<Produit>) {
    LazyColumn {
        items(produits) { p ->
            ProduitCard(p)    // réutilisée !
        }
    }
}
```

À retenir

- « Lazy » : seuls les éléments visibles sont composés
- items(...) : un composable par élément
- ProduitCard réutilisée telle quelle — la composition, c'est aussi la réutilisation
- Séance 5 : cette liste devient cliquable → écran de détail



Lire un layout XML

activity_main.xml — à savoir LIRE

```
<LinearLayout
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/tvNom"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button android:id="@+id/btnAjouter" .../>
</LinearLayout>
```

Table de correspondance mentale

- LinearLayout vertical ↔ Column · horizontal ↔ Row
- TextView ↔ Text · Button ↔ Button
- RecyclerView ↔ LazyColumn
- android:id + findViewById ↔ (plus besoin : la fonction reçoit ses données)
- match_parent ↔ fillMaxWidth / fillMaxSize

On ne l'écrit plus — on sait le lire. Et vous l'avez déjà lu : c'est le layout de CycleDeVie, séance 3.



Mini-TP 4 · « Faire vivre un écran »

1 Lire et prédire

Lire MainActivity.kt (carte statique, log RECOMP en place) ; prédire sur la feuille : lignes RECOMP au démarrage, puis après 3 clics.

2 TODO A — le compteur

remember + mutableStateOf + Button. Exécuter, cliquer 3 fois, compter les RECOMP au Logcat, expliquer l'écart.

3 TODO B — la carte sélectionnable

Second état booléen, Modifier.clickable, couleur qui change. La capture du Logcat prouve la recomposition.

4 Voie ouverte — juger une variante

UNE variante de mise en page demandée à l'IA ; en 3 lignes : laquelle garder, et pourquoi.

Dépôt en fin de séance : formulaire « S4 · Dépôt des livrables » — feuille, capture RECOMP, projet ZIP, jugement en 3 lignes.

